PROJECT SYNOPSIS & TECHNICAL SPECIFICATION (HINGLISH)

BHARAT YATRA

Smart Indian Tourist Cities Cost Calculator & AI Tour Planning Studio

Domain: Domestic Indian Tourism & Budgeting	**Stack:** Python / Streamlit / SQLite / Gemini AI	**Deployment:** Docker / Render Cloud (Live)

---

1. Project Overview & Problem Statement (Yeh Project Kyun Banaya?)

Purana Background vs Naya Mission: Yeh project originally ek US Government (GSA Per-Diem Rates & Chicago guidelines, USD $) travel cost calculator tha. Humne isko completely re-architect kiya aur specifically **Indian Domestic Tourism (INR Currency)** ke liye tailor-made banaya jisme pure India ke 45+ top tourist cities ka real dataset shaamil kiya gaya hai.

Real-World Problem: Jab bhi koi Indian traveler apni family ya friends ke sath trip plan karta hai (jaise Goa, Manali, Jaipur, Kerala), toh use alag-alag websites par jaakar hotels, trains/flights, daily food aur local cabs ka andaza lagana padta hai. Is fragmented process ki wajah se budget hamesha exceed ho jata hai ya trip plan adhoora reh jata hai.

Bharat Yatra ka Solution: Ek unified, modern web application jisme user ko single-click me complete itemized budget breakdown (Lodging, Food, Transit, Local commute, 10% Emergency buffer) milta hai, sath hi **Google Gemini 3.8 Flash AI** user ke personalized budget ke according day-wise itinerary generate karta hai.

2. Frontend vs Backend Complete Architecture Breakdown

Project ka architecture do primary layers me cleanly separated hai: **Frontend (UI / UX Layer)** jo user interaction handle karta hai, aur **Backend (Business Logic, Security, Data & AI Layer)** jo computational aur storage operations manage karta hai.

A. Frontend Layer (User Interface & Experience)

- **UI Framework: Streamlit** (Python-based modern reactive web UI framework) jo without heavy JS setup ke clean, fast web interfaces render karta hai.
- **Custom Design System & Styling:** Streamlit me custom CSS injection ke zariye **Nature-Toned Modern Palette** create kiya gaya hai (Deep Forest Green #1B4332, Sage Green #2D6A4F, Mint #52B788, aur Warm Sand cards).
- **Core UI Screens:**

1. *Authentication Portal:* Clean Login, Sign Up, aur Forgot Password OTP input forms.
2. *Cost Calculator Dashboard:* Dropdowns (Origin, Destination), Date Pickers, Number Inputs (Travelers, Nights), Budget Tier Radio Buttons.
3. *Visual KPI Cards:* Color-coded metric tiles showing Lodging, Food, Transit, Local Cab, Sightseeing, aur Emergency Buffer.
4. *AI Travel Studio Chat:* Interactive real-time chat interface (st.chat_message, st.chat_input) Copilot ke sath converse karne ke liye.
5. *Saved Trips Dashboard:* Past calculations ki list, detailed view, aur PDF download options.

- **Frontend Controller Files:** app.py (Main controller & dynamic page router) aur .streamlit/config.toml (Theme configuration).

B. Backend Layer (Logic, Security, Data & AI Intelligence)

Backend screen ke peeche ka saara mathematical calculation, database persistence, cryptographic security aur AI intelligence run karta hai. Yeh 5 core modules me divided hai:

Backend Module	Source File	Core Technology	Kaam / Responsibility in Project
Calculation Engine	src/travel_calculator.py	Python Core Math	Lodging, meals, intercity flights/trains, local cabs, aur 10% emergency buffer calculate karta hai. Indian numbering (Rs.) format karta hai.
Auth & Security	src/auth.py	SQLite3 + hashlib	Salted SHA-256 password hashing, user registration/login verify karna, aur 6-digit OTP generate/expire karna.
AI Agent Engine	src/ai_agent.py	Google Gemini 3.8 Flash	Gemini Cloud API ke sath communicate karta hai; trip context inject karke structured day-wise itinerary aur smart chat responses generate karta hai.
Email Dispatch	src/email_service.py	Resend REST API	Transactional email delivery handle karta hai; user ke verified inbox par instant 6-digit OTP deliver karta hai.
Data Processing	src/data.py + CSVs	Pandas Library	45+ Indian cities ke hotel rates, local transport rates aur ground distances ko query aur index karta hai.

C. Frontend aur Backend Aapas Mein Connect Kaise Hote Hain? (Data Flow)

1. **Reactive Event Triggering:** Jab user frontend screen (app.py) par koi button click karta hai (e.g. *Calculate Costs* ya *Login*), Streamlit synchronous function calls ke zariye backend module (src/) ko execute karta hai.

2. **Session State Persistence:** User ka login token, active trip metadata aur AI chat history memory me st.session_state ke zariye maintain rehti hai.

3. **Database & External API Sync:** Backend functions SQLite database se credentials/trips fetch karte hain aur external APIs (Google Gemini & Resend) se data process karke wapas frontend screen par dynamically render kar dete hain.

3. Technology Stack Specification (Kaunsi Technology Kyun Use Hui?)

Project me use kiye gaye sabhi major technologies, libraries aur unka specific role neeche table me detailed hai:

Component	Technology Used	Version	Role & Significance in Project
Web Framework	Streamlit	1.65.0+	Rapid reactive web UI, multi-page routing, state management
Language	Python	3.11 / 3.14	Core backend logic, data transformation, algorithms
Data Manipulation	Pandas	3.0.6+	CSV parsing, travel rates aggregation, distance indexing
Generative AI	Google GenAI SDK (Gemini 3.8 Flash)	2.21.0+	High-speed AI itinerary generation & dynamic copilot chat
Email Dispatch	Resend REST API	2.36.0+	Transactional email delivery for 6-digit password reset OTP
Database	SQLite3	Built-in	Zero-config relational database for users, OTPs, and saved trips
Cryptography	Python hashlib	Standard Lib	Salted SHA-256 password hashing for robust user security
Testing Suite	Pytest	9.1.1+	9 automated test cases covering math, validation, and currency
Containerization	Docker	Latest	Containerized image with dynamic $PORT binding for Render
Cloud Hosting	Render.com	Cloud PaaS	Automated GitHub CI/CD web service with secret env variables

4. Mathematical Budgeting Algorithm (Kharcha Calculate Karne Ka Formula)

Cost Calculation Engine transparent, deterministic aur mathematically verifiable formulas par operate karta hai:

```
Step 1: Per Person Cost Formula
Cost_Per_Pax = (Lodging_Rate * Nights / Pax_Per_Room) + (Daily_Food_Rate * Days) + Intercity_Transit +
(Local_Transit_Rate * Days) + Sightseeing_Fees

Step 2: Total Group Subtotal
Group_Subtotal = Cost_Per_Pax * Number_Of_Travelers

Step 3: 10% Emergency Buffer & Final Budget
Emergency_Buffer = Group_Subtotal * 0.10
Total_Trip_Cost = Group_Subtotal + Emergency_Buffer
```

5. Database Architecture & Relational Schema (SQLite3)

Application me 3 core relational tables hain jo complete user lifecycle aur trip history manage karte hain:

Table Name	Fields / Columns	Constraints & Security Purpose
users	id (INT PK), username (TEXT), email (TEXT UNIQUE), password_hash (TEXT), created_at (DATETIME)	User identification; passwords stored as salt$sha256_hash
otps	id (INT PK), email (TEXT), otp_code (TEXT), expires_at (REAL), is_used (INT)	Forgot password verification; 10 min expiry, invalidated upon use
saved_trips	id (INT PK), user_email (TEXT), origin (TEXT), destination (TEXT), days (INT), travelers (INT), tier (TEXT), total_cost (REAL), itinerary (TEXT), created_at (DATETIME)	Persistent storage for user's past calculations and AI itineraries

6. Project Directory & Architecture Breakdown

Project structure modular aur clean architecture follow karta hai, jisme concerns cleanly separated hain:

```
travel_cost_calculator/
|-- app.py                     # Frontend controller & multi-page navigation router
|-- requirements.txt           # Python dependencies (streamlit, google-genai, resend, etc.)
|-- Dockerfile                 # Container setup with dynamic $PORT binding for Render
|-- .env                       # API keys (GEMINI_API_KEY, RESEND_API_KEY)
|-- Bharat_Yatra_Synopsis.pdf  # Project formal synopsis in PDF format
|-- generate_pdf.py            # Automated ReportLab PDF generator script
|-- src/                       # Backend Source Code Modules
|   |-- auth.py                # User login, signup, SHA-256 hashing & SQLite queries
|   |-- email_service.py       # Resend REST API integration for email OTP delivery
|   |-- ai_agent.py            # Gemini 3.8 Flash prompt engineering & dynamic copilot
|   |-- travel_calculator.py   # Core math calculation engine & INR currency formatter
|   `-- data.py                # Fallback city master data & transport matrices
|-- data/                      # Backend Dataset Files
|   |-- cities_india.csv       # 45+ cities tier, hotel rates, and daily living costs
|   `-- ground_transport_rates.csv # Local transport matrices across Indian cities
`-- tests/                     # Automated Test Suites
    `-- test_calculator.py     # 9 Pytest test cases (currency, math, validation)
```

7. Cloud Deployment & DevOps Lifecycle (Render & GitHub)

Application ko continuous cloud deployment pipeline ke zariye live kiya gaya hai:

- **GitHub Repository:** Complete source code https://github.com/ShaikhVaiz/travel_cost_calculator.git par version controlled hai.
- **Dynamic Port Binding:** Render jaise cloud platforms par dynamic port allot hota hai. Dockerfile me hardcoded port ke bajaye --server.port=${PORT:-8501} configure kiya gaya hai.
- **Secrets Management:** Gemini API Key aur Resend API Key ko code me hardcode karne ke bajaye Render ke Environment Variables dashboard me securely inject kiya gaya hai.
- **Zero-Downtime Builds:** Jab bhi GitHub par naya commit push hota hai, Render automatically Docker image rebuild karke bina kisi downtime ke deployment live kar deta hai.

8. Viva & Interview Questions Guide (Hinglish Q&A;)

Examiners aur interviewers dwara puche jane wale top questions aur unke crisp answers:

Q1: Is project ka Frontend aur Backend alag kaise hai?

Ans: Frontend Streamlit aur custom CSS par based hai jo interactive UI, forms aur AI chat render karta hai. Backend Python modules par bana hai: calculation ke liye travel_calculator.py, SQLite database aur SHA-256 hashing ke liye auth.py, email OTP ke liye Resend API, aur AI generation ke liye Google Gemini 3.8 Flash.

Q2: Passwords ko database me kaise protect kiya gaya hai?

Ans: Plain text passwords bilkul store nahi hote. Har user ke liye unique cryptographic salt generate hota hai aur SHA-256 algorithm se hash karke SQLite me store kiya jata hai.

Q3: Forgot Password me OTP kaise dispatch hota hai?

Ans: Resend REST API SDK ke zariye user ke email par instant 6-digit verification code jata hai. Is code ki validity SQLite me 10 minute tak rehti hai aur ek baar verify hone par ye expire ho jata hai.

Q4: Gemini 3.8 Flash model kyun use kiya gaya?

Ans: Gemini 3.8 Flash ultra-fast inference speed deta hai jisse user ko bina kisi delay ke instant structured itinerary aur dynamic answers mil jate hain.

Q5: Render cloud deployment me kya challenge aaya tha?

Ans: Cloud platforms fixed port 8501 allow nahi karte. Humne Docker entrypoint me dynamic shell expansion ${PORT:-8501} use karke container ko dynamic port-aware banaya.